

Silent Failure Detection in Deep Neural Networks using Penultimate-Layer Embedding Geometry: Dispersion and Effective Rank as Training-Time Monitoring Signal

Chinmay Singh

Manipal Institute of Technology Bengaluru
Manipal Academy of Higher Education
Manipal 576104, India
chinmay2.mitblr2023@learner.manipal.edu

Amreen Ayesha

Manipal Institute of Technology Bengaluru
Manipal Academy of Higher Education
Manipal 576104, India
amreen.ayesha@manipal.edu

Abstract—When a neural network fails silently, its output looks fine. Accuracy holds and its confidence is high while something underneath has gone wrong. Monitoring metrics such as accuracy and Softmax confidence work at the output layer and are structurally unable to detect degradation in internal representations until it is already too late to intervene efficiently. This problem is referred to as silent failure. We study this problem in our work across four failure conditions: label noise, spurious correlations, distribution shift, and class imbalance. Instead of observing the output layer, we watch the geometry of penultimate-layer embeddings throughout training using two metrics: embedding dispersion, which measures how spread out the learned representations are in the latent space, and effective rank, which captures how variance is distributed across embedding dimensions using the Shannon entropy of the normalized singular value distribution. Neither metric requires gradient access or any change to the training procedure. Experiments are conducted on MNIST and CIFAR-10 across a simple CNN and ResNet-18, with five independent runs per condition for statistical reliability. Under label noise, effective rank was able to detect representational instability roughly 8.6 epochs before accuracy reflected it. The results are condition dependent and not uniform. Geometry provides the clearest early warning under label noise and spurious correlations, while under distribution shift and class imbalance all signals react simultaneously, which offered no timing advantage over accuracy. We report both findings in our work without making either look better than it is.

Keywords— *Deep neural networks, Representation learning, Out-of-distribution detection, Model monitoring, Embedding geometry, Effective rank, Training dynamics, Spurious correlations.*

I. INTRODUCTION

Models are being trained and tested rigorously to achieve better performance, but these models can fail silently as their internal representations degrade. Accuracy and Confidence stay up but it falls apart internally. This can go undetected, and by the time it is found the damage is usually done. We call this Silent Failure.

It can show up in at least four cases. Label noise, which can cause a model to fit corrupted targets while it still appears to converge[1]. The model can achieve the illusion of a higher accuracy using spurious correlations as a shortcut, but the illusion fails the moment the context is changed[2]. The model's internal representation can shift due to distribution changes while validation results remain the same. Severe class

imbalance lets the model ignore minority classes as those classes barely matter for the accuracy metric anyway.

In our work we will be using two geometry metrics extracted from the penultimate layer embeddings after each training epoch. The first is embedding dispersion, which will measure how spread out the representations are and second is effective rank[3], which tells us how information is distributed across different features. Using these we look to answer the question: can the geometry of learned representations tell us something is wrong before accuracy does? To which the answer we found was sometimes.

Our experiment showed that under Label Noise, Effective rank lead accuracy by 8.6 epochs in detecting the

first changes, which is a fairly large gap. However geometry gave no advantage under distribution shift and class imbalance as every single signal fired at epoch 1. Both results matter and we will be reporting it without making either look better.

The main contributions of our work are:

- Controlled evaluation of geometry metrics across four failure conditions, two datasets and two architectures with five independent runs per condition which is enough to report actual numbers.
- Lead time analysis with mean and standard deviation that runs across conditions. It tells us specifically when geometry monitoring helps and by how much.
- Providing evidence that effective rank is more consistent than dispersion as a monitoring signal as it has lower variance across all four conditions.

The remainder of this paper is organized as follows. Section II reviews the related literature and summarizes key prior work in Table I. Section III presents the methodology, which includes the proposed geometry metrics and lead-time formulation. Section IV discusses about the datasets and experimental setup. Section V presents the results across all four failure conditions. Section VI discusses the findings, and Section VII concludes the paper with limitations and directions for future work.

II. LITERATURE REVIEW

Table I summarises the papers referenced in this work along with their methodology, findings, and relevance to our approach.

TABLE I. LITERATURE REVIEW

Title	Authors & Year	Main Problem/Topic	Methodology / Key Contribution	Key Findings	Paper Summary
Effective Rank: Measure of Dimensionality	Roy & Vetterli (2007)	Measuring representation dimensionality	Defined effective rank using entropy of singular values	Captures intrinsic dimensionality of representations	Useful metric for analysing learned representations
Adam Optimizer	Kingma & Ba (2015)	Optimization in deep learning	Adaptive moment estimation	Fast and efficient convergence	Widely used optimizer
Dropout as Bayesian Approximation	Gal & Ghahramani (2016)	Uncertainty estimation	MC Dropout as Bayesian inference	Enables uncertainty estimation at inference	Practical approach to model uncertainty
Deep Residual Learning (ResNet)	He et al. (2016)	Training deep networks	Residual connections	Enables very deep networks	Foundation of modern architectures
On Calibration of Modern Neural Networks	Guo et al. (2017)	Model overconfidence	Temperature scaling for calibration	Modern networks are overconfident	Simple post-processing improves reliability
Baseline for OOD Detection	Hendrycks & Gimpel (2017)	Detecting OOD inputs	Softmax probability baseline	Lower confidence for OOD inputs	Simple baseline for anomaly detection
Deep Ensembles for Uncertainty	Lakshminarayanan et al. (2017)	Reliable uncertainty estimation	Ensemble of multiple models	Improves predictive uncertainty	Simple yet effective uncertainty estimation method
SVCCA Representation Analysis	Raghu et al. (2017)	Comparing neural representations	SVCCA method	Tracks representation similarity across layers	Useful for analysing training dynamics
Unified Framework for OOD Detection	Lee et al. (2018)	Improved OOD detection	Mahalanobis distance in feature space	Better separation of ID vs OOD data	Stronger detection than Softmax baseline
Intrinsic Dimension of Loss Landscapes	Li et al. (2018)	Complexity of optimization landscapes	Measured intrinsic dimensionality	Loss landscapes are low-dimensional	Insights into optimization behaviour
Shortcut Learning in Deep Neural Networks	Geirhos et al. (2020)	Models learning spurious correlations	Empirical analysis of shortcut behaviours in deep models	Models rely on shortcuts instead of meaningful features	Shows deep networks often learn unintended patterns
Energy-Based OOD Detection	Liu et al. (2020)	Robust OOD detection	Energy-based scoring functions	More stable OOD detection	Theoretically grounded alternative
Detecting Training Bugs in Deep Neural Networks	Zhang et al. (2020)	Identifying hidden bugs in training pipelines	Proposed techniques to detect silent failures during training	Training bugs can silently degrade model performance	Highlights importance of debugging training pipelines
Latent Space Geometry Graphs	Lassance et al. (2021)	Improving robustness	Graph-based latent space constraints	Improves model robustness	Alters training objective
Low-Rank Bias in Deep Networks	Huh et al. (2023)	Representation structure	Empirical study of rank properties	Networks favour low-rank representations	Supports dimensionality reduction trends
Silent Failures in Medical Models	Roschewitz et al. (2023)	Undetected failures in models	Visualization tools for latent space	Confidence fails to detect issues	Need for better failure detection methods

TABLE 1 Summary of related literature referenced in this work.

A. Confidence and Uncertainty Estimation

Guo et al.[4] found that most modern models tend to be overconfident with unreliable predictions. Their work showed that temperature scaling can fix this post training. On the other hand MC Dropout [5] and deep ensembles[6] approach this differently by estimating uncertainty at inference time. These techniques are genuinely useful but only watch the output layer leaving the degrading representations out of the picture. We show this directly by our spurious features condition. Confidence climbs up steadily throughout training as the

representations becomes more dependent on a small colour patch that does not exist in the test set.

B. Out-of-Distribution Detection

Hendrycks and Gimpel [7] approached the problem of unreliable predictions by showing that raw Softmax scores can be used as a baseline to flag inputs that look different from training data. Lee et al. [8] went further as he argued that Mahalanobis distance captures distributional differences more precisely in feature space. A theoretical cleaner alternative is Energy-based scores [9]. However all these methods work at inference time on individual inputs. They are not designed to

detect a model that is degrading on inputs it can still recognise. Our work focus exactly on this, failures that are invisible during inference-time checks because the inputs appear fine.

C. Shortcut Learning

Geirhos et al. [2] stated that deep networks do not always learn what we intend them to. They often take shortcuts or cheat by exploiting spurious patterns which correlates with the label even when a more meaningful feature is present. The representations structure reveal the dependency on shortcuts before it shows in the accuracy drop. we tested this by setting up a spurious features condition and found that Dispersion fired 0.2 epochs before accuracy and confidence kept rising up through training signalling that the model was improving but becoming dependent on unreliable patterns.

D. Representation Analysis

A hefty amount of research has been done on what the internal representations look like during and after training. Raghu et al. [10] came up with SVCCA as a method to compare the representations across layers and at different training stages. Li et al. [11] worked on intrinsic dimensionality of loss landscapes. Roy and Vetterli [3] defined effective rank which is the exponential of Shannon entropy of normalized singular values of a matrix. We use this in our work. Recently Huh et al. [12] found that deep networks lean towards low-rank representations, which aligns with our observations: changes in effective rank during training can reveal different failure modes.

E. Silent Failure Detection

There has been similar work done compared to ours. Roschewitz et al. [13] worked on silent failure in medical image classification. He found that confidence did not catch the problem, to which his response was SF visuals, an interactive tool for visualizing the latent space structure after training. Zhang et al. [1] addressed a different problem entirely. He traced the problem down to bugs and errors in training pipelines, but this is not what we are studying. Lassance et al. [15] proposed constraining the latent geometry graphs as part of training. This was done to increase robustness, but it also meant changing the objective. Our approach proposes scalar metrics, which can monitor continuously during training. This does not tamper with the training process.

III. METHODOLOGY

A. Setup

Let $f_t(\cdot)$ be the function that maps an input x to its penultimate-layer embedding under the model parameters at epoch t . For a dataset $D = \{x_i\}$ with N samples, the embedding matrix at epoch t is $Z^n \in \mathbb{R}^{N \times d}$ using which we compute two scalar metrics from Z^n at each epoch and track their route over training.

B. Embedding Dispersion

Dispersion measures the average distance of embeddings from their centroid, as defined in (1). Here u^n is the mean embedding over the test set. When dispersion collapses, the representations cluster together mainly because the model cheats by depending on low-dimensional shortcuts, whereas when it rises sharply, mismatched supervision signals cause the embedding space to spread in different directions.

$$Dispersion(Z^n) = \left(\frac{1}{N}\right) \sum_i \|z_i^n - \mu^n\|_2 \quad (1)$$

C. Effective Rank

Dispersion tells us how spread apart the data points are from each other, but not how that spread is organized across dimensions. For that we use effective rank [3]. Let $\{s_i\}$ be the singular values of Z^n and $p_i = \frac{s_i}{\sum_j s_j}$ the normalized distribution. Effective rank is defined in (2) and describes how many dimensions meaningfully contribute to the representation. A high effective rank shows that variance is spread evenly across dimensions whereas a low effective rank shows that only a few dimensions contribute because it depends on normalized singular values. Effective rank is not impacted by the scaling of dimensions therefore making it more stable during training.

$$ERank(Z^n) = \exp\left(-\sum_i p_i \log p_i\right) \quad (2)$$

D. Lead-Time Metric

To measure how early a signal react, we look for the first epoch where it moves meaningfully. For a normalised signal $s(t)$, the first-change epoch is the smallest $t > 0$ such that $|s(t) - s(0)| > \tau$, where $\tau = 0.1$ this threshold is fixed and identical across all conditions. The geometry Δ lead is then defined in (3), where a positive Δ indicates that the geometric signal changes before accuracy. We run five independent seeds per condition and report the mean and standard deviation of Δ .

$$\Delta = t_{acc} - t_{geom} \quad \begin{matrix} \text{(positive } \Delta \\ = \text{geometry fires first)} \end{matrix} \quad (3)$$

IV. DATASET AND SETUP

A. Failure Conditions

Label Noise (MNIST). 30% of the training labels are randomly changed before training. The model can still learn enough to score reasonably on the corrupted training set, so accuracy does not immediately signal anything is wrong, however the representations have to accommodate and handle conflicting labels. MNIST:

<http://yann.lecun.com/exdb/mnist>

Spurious Features (CIFAR-10). A small colour patch is added to 90% of training images, with the hue being correlated to the class label. At test time the patch is removed to let the model learn a feature that is perfectly predictive in training and completely useless at evaluation, yet accuracy stays high because it has also picked up enough real signal to classify correctly on the balanced test set. CIFAR-10:

<https://www.cs.toronto.edu/~kriz/cifar.html>

Distribution Shift (CIFAR-10 / ResNet-18). The model is trained on clean CIFAR-10 data. Gaussian noise with $\sigma = 0.1$ is added at each evaluation before extracting embeddings to introduce a controlled shift between training and evaluation distributions. Accuracy may remain stable despite the mismatch in input distribution.

Class Imbalance (CIFAR-10). Airplane and automobile combined make up 90% of the training set and the other eight classes split the remaining 10%. The test set is balanced across all ten classes. Overall accuracy stays high as two classes dominate, however the minority class representations have almost no training signal to build upon.

B. Architectures and Protocol

For MNIST and most CIFAR-10 experiments, we use a straightforward CNN: two convolutional layers, max pooling, and a 256-unit fully connected penultimate layer. For the distribution shift condition, we switch to ResNet-18 [16] with a 512-unit penultimate layer, partly to check whether the geometry signals also appear in a significantly deeper architecture. All models are trained for 20 epochs using Adam [17] with a learning rate of 0.001 and batch size of 256. Every condition is run five times with different random seeds. No early stopping. No geometry-aware regularization of any kind.

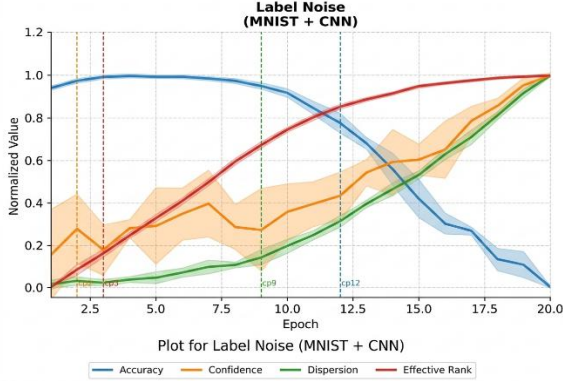


Fig. 1. Normalised signal trajectories for Label Noise (MNIST + CNN). Shaded bands show ± 1 std across five runs. Dashed lines mark first-change epochs. Blue = Accuracy, Orange = Confidence, Green = Dispersion, Red = Effective Rank.

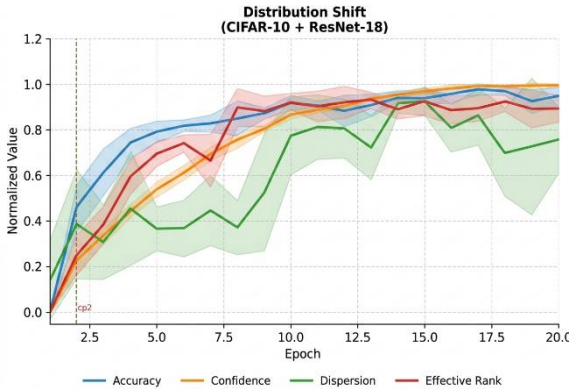


Fig. 3. Normalised signal trajectories for Distribution Shift (CIFAR-10 + ResNet-18). Shaded bands show ± 1 std across five runs. Dashed lines mark first-change epochs. Blue = Accuracy, Orange = Confidence, Green = Dispersion, Red = Effective Rank.

A. Label Noise (MNIST)

This is where the geometry advantage is most obvious, as seen in Fig. 1. Effective rank, computed using (2), crossed the detection threshold at epoch 1.8 (± 0.40) on average.

V. RESULTS

Table II shows the first-change epoch (mean and std across 5 runs) for each signal under each condition. Figs. 1–4 show the full signal trajectories for each failure condition with shaded standard deviation bands.

Table III. Results. First-change epoch (mean $\pm$ std, 5 runs). Lower values mean the signal reacted sooner. Delta is the geometry lead over accuracy in epochs; positive means geometry fired first

Condition	Acc.	Conf.	Disp.	E.Rank	ΔD	ΔR
Label Noise (MNIST)	10.4 ± 0.49	1.2 ± 0.40	7.2 ± 1.94	1.8 ± 0.40	+3.2	+8.6
Spurious Features (CIFAR-10)	1.2 ± 0.40	1.6 ± 0.80	1.0 ± 0.00	2.6 ± 0.49	+0.2	-1.4
Dist. Shift (ResNet-18)	1.0 ± 0.00	1.0 ± 0.00	1.4 ± 0.80	1.0 ± 0.00	-0.4	0.0
Class Imbalance (CIFAR-10)	1.0 ± 0.00	1.2 ± 0.40	2.6 ± 0.49	1.0 ± 0.00	-1.6	0.0

Table III. Acc. = Accuracy, Conf. = Confidence, Disp. = Dispersion, E.Rank = Effective Rank, ΔD = Geometry lead of Dispersion over Accuracy (epochs), ΔR = Geometry lead of Effective Rank over Accuracy (epochs).

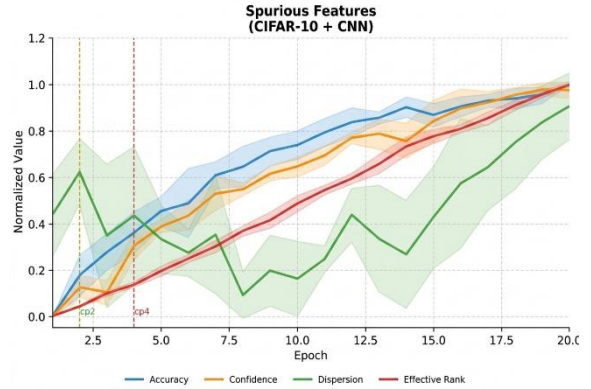


Fig. 2. Normalised signal trajectories for Spurious Features (CIFAR-10 + CNN). Shaded bands show ± 1 std across five runs. Dashed lines mark first-change epochs. Blue = Accuracy, Orange = Confidence, Green = Dispersion, Red = Effective Rank.

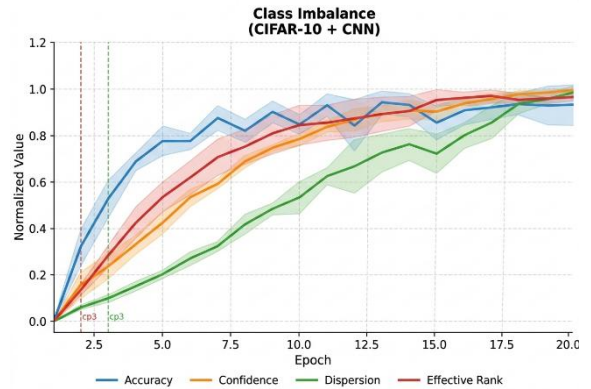


Fig. 4. Normalised signal trajectories for Class Imbalance (CIFAR-10 + CNN). Shaded bands show ± 1 std across five runs. Dashed lines mark first-change epochs. Blue = Accuracy, Orange = Confidence, Green = Dispersion, Red = Effective Rank.

Accuracy did not move until epoch 10.4 (± 0.49). That is a gap of 8.6 epochs. Dispersion, computed using (1), was slower, firing at 7.2 (± 1.94), but still 3.2 epochs ahead of accuracy. Looking at Fig. 1, the accuracy line stays near its peak for roughly the first half of training while both geometry

signals are already reacting. The output layer does not give any indications of the unstable representations underneath however geometry does.

B. Spurious Features (CIFAR-10)

Everything here happens quickly, as shown in Fig. 2. Dispersion fires at epoch 1.0 (± 0.00), just ahead of accuracy at 1.2 (± 0.40). Effective rank is actually slower than accuracy, firing at 2.6 (± 0.49) and behind by 1.4 epochs. So the geometry lead-time story is weak here. What is more interesting is confidence: it registers at 1.6 and then just keeps rising for the entire 20 epochs. Every other signal stabilises. Confidence does not. The model is growing more certain about predictions that depend on a patch of colour that disappears at test time. It is not a calibration issue in the usual sense; the model found something that works perfectly in training and is completely confident about it. The problem is that what it found is not the thing we wanted it to find.

C. Distribution Shift (CIFAR-10 / ResNet-18)

Every signal hit the threshold by epoch 1.4, as shown in Fig. 3. Accuracy and effective rank both registered at 1.0 (± 0.00); dispersion followed at 1.4 (± 0.80). Geometry provides no lead-time advantage here at all. The noise perturbation is apparently large enough that the shift is immediately visible at every level of the network. What is worth noting is the width of the dispersion band in Fig. 3. It stays wide across all 20 epochs, which suggests the representations are never really settling, even after accuracy has stabilized. That ongoing instability does not appear in the accuracy curve.

D. Class Imbalance (CIFAR-10)

Accuracy and effective rank both fire at epoch 1.0 (± 0.00), as shown in Fig. 4. Dispersion follows at 2.6 (± 0.49), trailing accuracy by 1.6 epochs. No geometry advantage. What accuracy actually measures here is worth noting: it stays high because airplane and automobile, the two majority classes, are well represented and easy to classify. The eight minority classes barely get enough training examples to form coherent representations, and their embeddings collapse almost immediately. You can see this in Fig. 4 as dispersion dropping and flattening early. Since those classes only represent a small fraction of the test set, the accuracy metric never reflects it. Geometry confirms the collapse but cannot get there before accuracy does.

E. Cross-Condition Summary

Across all four conditions, effective rank was more stable than dispersion (lower std in Table I). If we had to pick one metric to deploy, it would be effective rank. The bigger pattern is about when geometry monitoring is useful at all: it helps when representational damage accumulates gradually while output metrics are slow to respond. Under label noise, that gap is large and consistent across runs. Under spurious features, the dispersion lead is small, but the confidence finding is arguably more useful. Under distribution shift and class imbalance, the gap disappears entirely.

VI. DISCUSSION

A. When Geometry Helps Most

The clearest result we have is under label noise where effective rank detected representational instability at epoch 2 in conditions where accuracy remained stable until epoch 10. That 8.6-epoch window is real and practically useful. An

operator who checked effective rank at epoch 5 would have known something was wrong whereas an operator who checked only accuracy would have only seen a well-behaved training run. When failure is silent and gradual it is then that this difference matters most, which is why it is the hardest to catch any other way.

B. When Geometry Does Not Help

Geometry does not give a clear advantage in every condition, and it is important to highlight this. Under distribution shift and class imbalance, every signal crossed the detection threshold at epoch 1. Nothing about these failures was gradual and they were immediately visible everywhere including at the output layer. Geometry did not lead in any way. We think this is actually an important finding as it tells us something about the nature of the failure. When degradation is sudden and total, no monitoring signal gets there first. The cases where geometry genuinely helps are the ones where representational damage piles up slowly, while the output layer stays unaware of these changes. They are one of the more dangerous cases in practice because they are harder to catch.

C. Confidence as a False Signal

Confidence gave us some interesting observations, it behaved roughly together with accuracy under label noise, both reacted slow and eventually reflected the problem, but under spurious features, confidence went up. The model grew more certain in its predictions throughout the entire training run while at the same time it was becoming more dependent on a patch of colour that would disappear at test time. It is a case of confidence actively pointing the wrong way making it uninformative. The reason is Softmax confidence tracks how much one logit dominates the others and not whether the features driving that logit are stable or meaningful. A model exploiting a reliable shortcut may be highly confident, even though that shortcut may fail at deployment.

VII. CONCLUSION AND FUTURE WORK

Geometry metrics worked well in giving early warning signals only in some conditions. But these are the conditions where silent failure is hardest to detect. So the question we set out to ask whether the geometry of penultimate-layer embeddings can give earlier warning of silent failure than accuracy or confidence does. The answer is sometimes.

Under label noise, effective rank detected structural failure at epoch 2, whereas accuracy held steady until epoch 10, a lead of 8.6 epochs. Under spurious features, dispersion got there marginally first, but the more useful observation is about confidence: it climbed throughout training while the representations were becoming more shortcut-dependent, actively misrepresenting the state of the model. Under distribution shift and class imbalance, all signals fired at epoch 1 and geometry offered no advantage.

We report the null results as directly as the positive ones because the distinction matters. Geometry monitoring is not a universal upgrade. Its value is specifically tied to whether representational degradation is gradual or abrupt. When it is gradual, effective rank proved to be a low-overhead early warning signal that required no changes to training at all. Across our experiments, effective rank was consistently more reliable than dispersion, with lower run-to-run variance in all four conditions.

When working with noisy labels or datasets prone to spurious correlations, tracking effective rank only adds minimal overhead and based on our results, can help surface representational problems well before they reach the output layer in practice. The open questions are about scale, threshold selection, and what to actually do once the warning fires. Those feel like the right next problems.

There are a few things worth being upfront about. These experiments are limited to image classification on MNIST and CIFAR-10, which were chosen to be small enough for fast training and reproducible results but may not represent real deployment scale. Whether the geometry signals we observe survive at larger scale with bigger models, larger datasets, and longer training runs remains an open question. The threshold $\tau = 0.1$ was set for convenience and kept fixed across all conditions; a better way to choose this in practice would be needed for real deployment. Finally, geometry metrics tell us something changed but not what changed or why. They are a monitoring tool, not a diagnostic one. These are the directions we think the work should go next.

REFERENCES

- [1] Y. Zhang et al., "Detecting training bugs in deep neural networks," in Proc. ICSE, 2020.
- [2] R. Geirhos et al., "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, no. 11, pp. 665-673, 2020.
- [3] O. Roy and M. Vetterli, "The effective rank: A measure of effective dimensionality," in Proc. EUSIPCO, 2007, pp. 606-610.
- [4] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. ICML, 2017, pp. 1321-1330.
- [5] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation," in Proc. ICML, 2016, pp. 1050-1059.
- [6] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in Proc. NeurIPS, 2017.
- [7] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples," in Proc. ICLR, 2017.
- [8] K. Lee et al., "A simple unified framework for detecting out-of-distribution samples and adversarial attacks," in Proc. NeurIPS, 2018.
- [9] W. Liu, X. Wang, J. Owens, and Y. Li, "Energy-based out-of-distribution detection," in Proc. NeurIPS, 2020.
- [10] M. Raghu, J. Gilmer, J. Yosinski, and J. Sohl-Dickstein, "SVCCA," in Proc. NeurIPS, 2017.
- [11] C. Li, H. Farkhoor, R. Liu, and J. Yosinski, "Measuring the intrinsic dimension of objective landscapes," in Proc. ICLR, 2018.
- [12] M. Huh, M. Maire, and B. A. Olshausen, "The low-rank simplicity bias in deep networks," TMLR, 2023.
- [13] M. Roschewitz, G. Khara, and B. Glocker, "Understanding silent failures in medical image classification," in Proc. MICCAI, 2023.
- [14] C. Lassance, V. Gripon, and A. Ortega, "Representing deep neural networks latent space geometries with graphs," *Algorithms*, vol. 14, no. 2, 2021.
- [15] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. CVPR, 2016, pp. 770-778.
- [16] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. ICLR, 2015.